

- **most_frequent**
 - Uvijek predviđa najčešću klasu u trening skupu
 - Korisno za neizbalansirane skupove (npr. 90% klasa A)
 - Baseline za accuracy → većina modela mora biti bolja od toga
 - Primjer: Ako su klase [0, 0, 0, 1, 0], model uvijek predviđa 0
- **prior**
 - Isti rezultat kao **most_frequent**, ali koristi raspodjelu klase (Y) bez eksplicitnog promatranja podataka (X)
- **stratified**
 - Predikcije se nasumično biraju prema stvarnom omjeru klase u trening skupu
 - Npr. ako trening ima 70% klasa A i 30% klasa B, onda će se predikcije nasumično birati tim omjerom
- **uniform**
 - Predikcije se biraju potpuno nasumično, sa istom vjerojatnošću za sve klase
 - Idealno kao najniži mogući baseline → očekuje se da ga svi modeli nadmaše
 - Ako imaš 3 klase, svaka se bira s vjerojatnošću 1/3
- **constant**
 - Uvijek predviđa fiksnu klasu koju sam navedeš putem parameta constant=...
 - Korisno kad previše ručno testiraš koliko je točan model ako se "zaglavio" u jednoj klasi



The screenshot shows a Google Colab notebook with the following content:

```

from sklearn import metrics, dummy
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
tvs = train_test_split(iris.data, iris.target,
                        test_size=0.3, random_state=0)
iris_train_ftvs, iris_test_ftvs,
iris_train_tgt, iris_test_tgt = tts

baseline = dummy.DummyClassifier(strategy="most_frequent")
baseline.fit(iris_train_ftvs, iris_train_tgt)
base_pred = baseline.predict(iris_test_ftvs)
base_acc = metrics.accuracy_score(base_pred, iris_test_tgt)
print(base_acc)

0.3

import pandas as pd
strategies = ['constant', 'uniform', 'stratified',
             'prior', 'most_frequent']
# set up args to create different DummyClassifier strategies
baseline_args = [{"strategy":s} for s in strategies]

```

The notebook interface includes a top bar with the file name "06_01_01_osnovny_dummy_klassifikator.ipynb", tabs for "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help", and a command bar with options "Code", "Text", "Run all", and a RAM usage indicator showing "RAM 0MB". The bottom status bar shows "Variables", "Terminal", "S: 002 PM", and "Python 3".



The screenshot shows a Google Colab notebook with the following Python code:

```
import testapp
from sklearn.metrics import get_scorer_names
testapp.fill(str(sorted(get_scorer_names())))

[{'accuracy', 'adjusted_mutual_info_score', 'adjusted_rand_score',
 'average_precision', 'balanced_accuracy', 'completeness', 'f1',
 'f1_macro', 'f1_micro', 'f1_samples', 'f1_weighted', 'fowlkes_mallows_score',
 'homogeneity_score', 'jaccard', 'jaccard_macro', 'jaccard_micro',
 'jaccard_samples', 'jaccard_weighted', 'matthews_correlation',
 'mutual_info_score', 'neg_brier_score', 'neg_log_loss',
 'neg_mean_absolute_error', 'neg_mean_absolute_error',
 'neg_mean_absolute_percentage_error', 'neg_mean_gamma_deviance',
 'neg_mean_poisson_deviance', 'neg_mean_squared_error',
 'neg_mean_squared_log_error', 'neg_mean_squared_error',
 'neg_negative_loglikelihood_ratio', 'neg_root_mean_squared_error',
 'neg_root_mean_squared_log_error', 'normalized_mutual_info_score',
 'positive_likelihood_ratio', 'precision', 'precision_macro',
 'precision_micro', 'precision_samples', 'precision_weighted', 'r2',
 'rand_score', 'recall', 'recall_macro', 'recall_micro',
 'recall_samples', 'recall_weighted', 'roc_auc', 'roc_auc_ov',
 'roc_auc_ov_weighted', 'roc_auc_ov_weighted',
 'top_k_accuracy', 'v_measure_score'}
```

	<i>I think: pot is hot</i>	<i>I think: pot is cold</i>
pot is hot	I thought hot it is hot I'm right	I thought cold it isn't cold I'm wrong
pot is cold	I thought hot it isn't hot I'm wrong	I thought cold it is cold I'm right



6.2.1 Matrice konfuzije (2/2)



	<i>I think: pot is hot (Positive)</i>	<i>I think: pot is cold (Negative)</i>
pot is hot	True (predicted) Positive	False (predicted) Negative
pot is cold	False (predicted) Positive	True (predicted) Negative

	Predicted Positive (PredP)	Predicted Negative (PredN)
Real Positive (RealP)	True Positive (TP)	False Negative (FN)
Real Negative (RealN)	False Positive (FP)	True Negative (TN)



6.2.3 Mjerni podaci iz matrice konfuzije (osjetljivost, sensitivity (recall).)

	Predicted Positive (PredP)	Predicted Negative (PredN)
Real Positive (RealP)	True Positive (TP)	False Negative (FN)
Real Negative (RealN)	False Positive (FP)	True Negative (TN)

$$\frac{TP}{TP+FN} = \frac{TP}{\text{RealP}}$$

$$\frac{TP}{TP+FN} + \frac{FN}{TP+FN} = \frac{TP+FN}{TP+FN} = 1$$

6.2.3 Mjerni podaci iz matrice konfuzije (specifičnost (specifity), preciznost (precision).)

	Predicted Positive (PredP)	Predicted Negative (PredN)
Real Positive (RealP)	True Positive (TP)	False Negative (FN)
Real Negative (RealN)	False Positive (FP)	True Negative (TN)

$$\frac{TN}{FP+TN} = \frac{TN}{\text{RealN}}$$

$$\frac{TP}{\text{PredP}} = \frac{TP}{TP+FP}$$

		Predicted		
		P	N	
Actual	P	TP	FN	TP / (TP + FN) sensitivity recall, TPR
	N	FP	TN	TN / (FP + TN) specificity TNR
		TP / (TP + FP) precision		

6.2.4 Kodiranje matrice konfuzije

		Predicted Values	
		Positive	Negative
Actual Values	Positive	TP	FN
	Negative	FP	TN

Broj cipela	Spol
36	F
37	F
38	F
39	F
40	F
41	M
42	M
43	M
44	M
45	M
38	F
39	F
39	M
42	M
42	F



Broj cipela	Spol
36	F
37	F
38	F
39	F
40	F
41	M
42	M
43	M
44	M
45	M
38	F
39	F
39	M
42	M
42	F

`train_test_split(X,y,test_size=0.66,
random_state=41,stratify=y)`

Broj cipela	Spol	Predikcija
36	F	F
37	F	
38	F	F
39	F	F
40	F	F
41	M	F
42	M	M
43	M	M
44	M	
45	M	
38	F	
39	F	
39	M	F
42	M	M
42	F	M

GRANICA ? IZMEĐU 41 i 42!

$\begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix}$

POKAZATELJI

- ACCURACY – 0.7
- RECALL – 0.6
- PRECISION – 0.75
- SPECIFICITY – 0.8
- $F1 - 2 \times (0.75 \times 0.6) / (0.75 + 0.6) = 0.667$

		Predicted		
		P	N	
Actual	P	TP	FN	TP / (TP + FN) sensitivity recall, TPR
	N	FP	TN	TN / (FP + TN) specificity TNR
		TP / (TP + FP) precision		

$\begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix}$ Google Colab

```

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score, classification_report)

data = pd.DataFrame({
    "shoe_size_eu": [36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 38, 39, 39, 42, 42],
    "gender": ["F", "F", "F", "F", "F", "M", "M", "M", "M", "M", "F", "M", "M", "F", "F"]
})

X = data["shoe_size_eu"].values
y = data["gender"].values

knn = KNeighborsClassifier(n_neighbors=3)
  
```

POKAZATELJI

- ACCURACY – 0.7
- RECALL – 0.8
- PRECISION – 0.667
- SPECIFICITY – 0.6
- $F1 - 2 \times (0.8 \times 0.667) / (0.8 + 0.667) = 0.727$

		Predicted		
		P	N	
Actual	P	TP	FN	TP / (TP + FN) sensitivity recall, TPR
	N	FP	TN	TN / (FP + TN) specificity TNR
		TP / (TP + FP) precision		

[[4 1]
[2 3]] 

```

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score, classification_report)

data = pd.DataFrame({
    "shoe_size_eu": [36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 38, 39, 39, 42, 42],
    "gender": ["F", "F", "F", "F", "F", "M", "M", "M", "M", "M", "F", "M", "M", "F", "F"]
})

X = data["shoe_size_eu"].values
y = data["gender"].values

knn = KNeighborsClassifier(n_neighbors=3)
  
```

6.2.6 F1

$$F_1 = 2 \times \frac{1}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}}$$

$$F_1 = 2 \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

Google
colab

classification_report

```
labels=['M', 'F']
[[3 2]
 [1 4]]
accuracy : 0.7000
precision: 0.7500
recall   : 0.6000
f1       : 0.6667
```

```
=== Classification Report ===
              precision    recall  f1-score   support

 F   0.6667    0.8000    0.7273         5
 M   0.7500    0.6000    0.6667         5

 accuracy          0.7000         10
 macro avg         0.7083    0.7000    0.6970         10
 weighted avg      0.7083    0.7000    0.6970         10
```

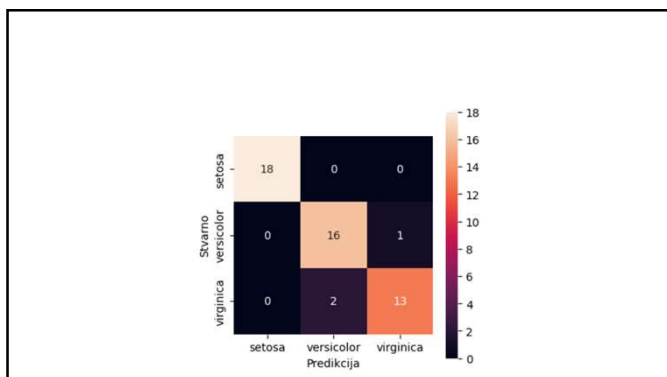
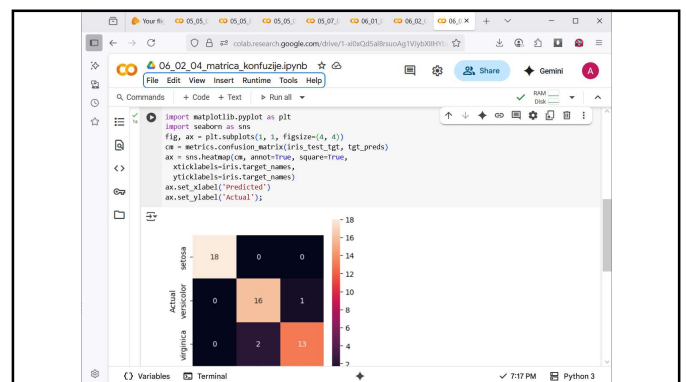
Google
colab

```
from sklearn import neighbors
from sklearn import metrics
from sklearn.model_selection import train_test_split

from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data
y = iris.target

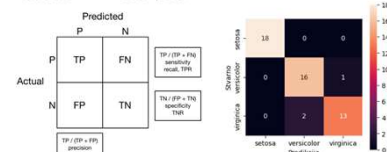
iris_train_ftrs, iris_test_ftrs, iris_train_tgt, iris_test_tgt = train_test_split(X, y, test_size=1/3, random_state=0)
tgt_preds = neighbors.KNeighborsClassifier().fit(iris_train_ftrs, iris_train_tgt).predict(iris_test_ftrs)
print("accuracy:", metrics.accuracy_score(iris_test_tgt, tgt_preds))
cm = metrics.confusion_matrix(iris_test_tgt, tgt_preds)
print("confusion matrix:", cm, sep="\n")
```

accuracy: 0.94
confusion matrix:
[[18 0 0]
 [0 16 1]
 [0 2 13]]



6.2.5 Rad s višestrukim klasama:

$$\frac{TP}{PredP} = \frac{TP}{TP+FP}$$

Google
colab

```

from sklearn import neighbors
from sklearn import metrics
from sklearn.model_selection import train_test_split

from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data
y = iris.target

iris_train_ftrs, iris_test_ftrs, iris_train_tgt, iris_test_tgt = train_test_split(X,
y, test_size=1/3, random_state=2)
import numpy as np
tgt_preds = neighbors.KNeighborsClassifier().fit(iris_train_ftrs, iris_train_tgt).predict(iris_test_ftrs)
macro_prec = metrics.precision_score(iris_test_tgt,
tgt_preds,
average='macro')
print("macro:", macro_prec)
cm = metrics.confusion_matrix(iris_test_tgt, tgt_preds)
n_labels = len(iris.target_names)
print("should equal 'macro avg.':",
# correct column # columns

```

$(\text{np.diag(cm)} / \text{cm.sum(axis=0)}).sum() / \text{n_labels})$

- $(\text{np.diag(cm)} / \text{cm.sum(axis=0)}).sum() / \text{n_labels})$
 - np.diag(cm)
 - Uzima dijagonalu matrice cm → to su točno klasificirani primjeri za svaku klasu.
 - Ako je cm $n \times n$, onda np.diag(cm) daje vektor duljine n .
 - cm.sum(axis=0)
 - Zbroj po stupcima matrice → ukupan broj predviđanja za svaku klasu.
 - Dakle: koliko je puta model predvidio pojedinu klasu (bez obzira je li pogodio ili ne).
- $\text{np.diag(cm)} / \text{cm.sum(axis=0)}$
 - Za svaku klasu: ispravna predviđanja za klasu/sva predviđanja te klase
 - To je zapravo precision (preciznost) za svaku klasu..
- $\text{sum()} / \text{n_labels}$
 - Računa prosjek preciznosti po klasama (macro-averaged precision).

```

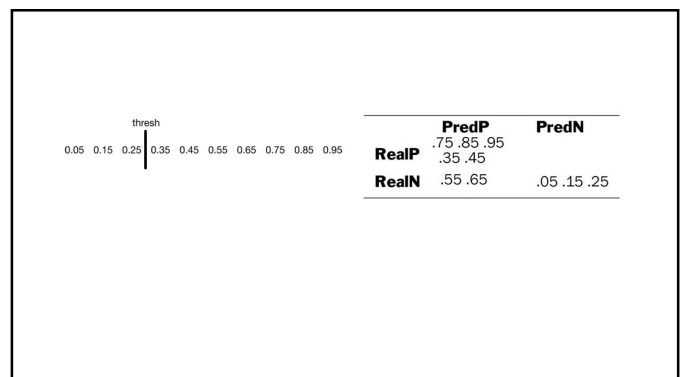
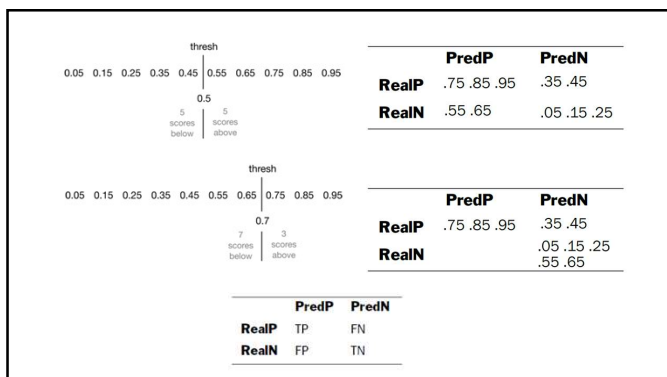
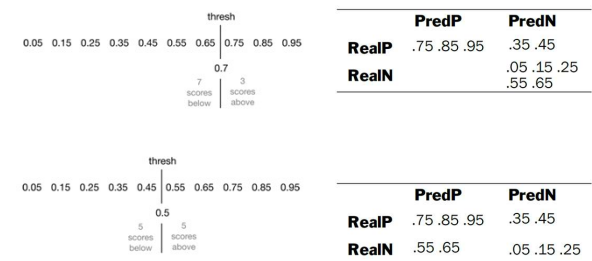
print("macro:", metrics.precision_score(iris_test_tgt,
tgt_preds,
average='macro'))
cm = metrics.confusion_matrix(iris_test_tgt, tgt_preds)
print("should equal 'macro'=",
# TP.sum() / (TP+FP).sum() -> {
# all correct / all preds
np.diag(cm).sum() / cm.sum()})

[ ] print(metrics.classification_report(iris_test_tgt, tgt_preds))
cm = metrics.confusion_matrix(iris_test_tgt, tgt_preds)
print("row counts equal support": cm.sum(axis=1))

precision recall f1-score support
0 1.00 1.00 1.00 16
1 0.89 0.94 0.91 17
2 0.93 0.87 0.90 15

```

6.3 ROC krivulje



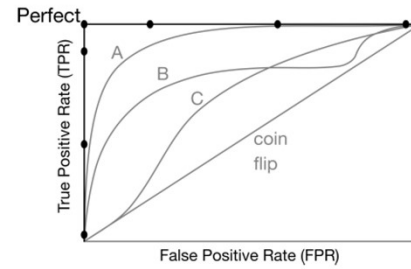


$$TPR = \frac{TP}{TP+FN} = \frac{TP}{\text{RealP}}$$

$$\frac{TN}{FP+TN} = \frac{TN}{\text{RealN}}$$

$$FPR = 1 - \frac{TN}{FP+TN}$$

6.3.1 Uzorci u ROC-u



Broj cipela	Spol
36	F
37	F
38	F
39	F
40	F
41	M
42	M
43	M
44	M
45	M
38	F
39	F
39	M
42	M
42	F



Google
colab

```
[14]: from sklearn.metrics import roc_curve, roc_auc_score
import numpy as np

y_pred_stat = knn.predict_proba(X_test[:, 1])
print(y_pred_stat)

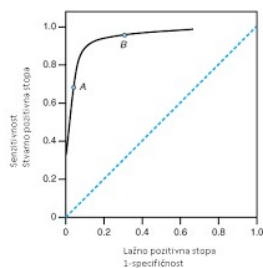
fpr, tpr, thresholds = roc_curve(y_test, y_pred_stat, pos_label='M')

roc_matrix = np.column_stack((fpr, tpr, thresholds))
print("Roc matrix (prvih 5 redaka):")
print(roc_matrix[:5])

# Testni kod za igru da dobijemo prag za željeni target_recall
target_recall = 0.9
idx = np.where(tpr >= target_recall)[0]
thr_for_recall = thresholds[idx[0]] if len(idx) else 0.5
print(thr_for_recall)

y_pred_custom = (y_pred_stat >= thr_for_recall).astype(int)
print(y_pred_custom)
print(np.where(y_test == "M", 1, 0))
```

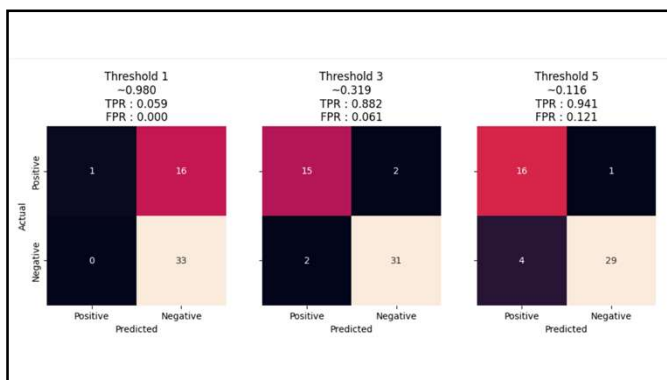
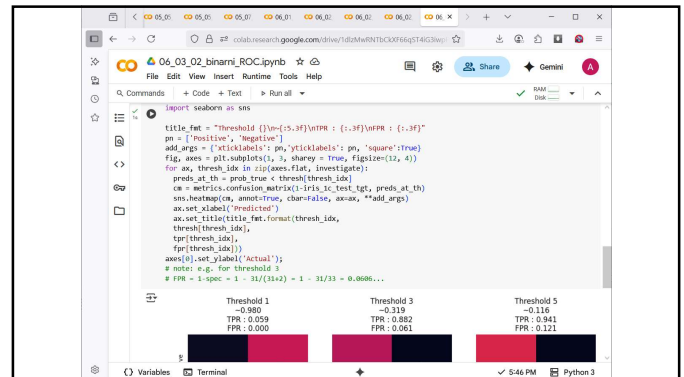
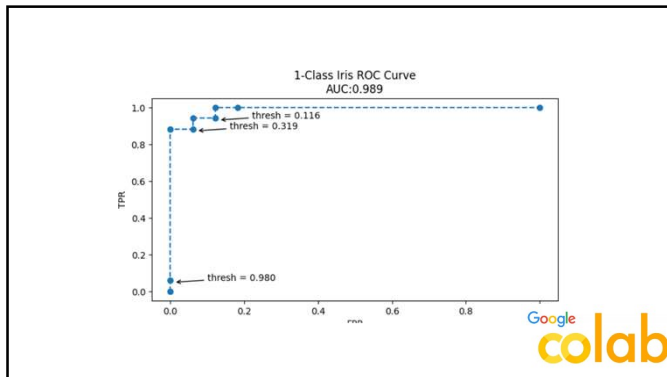
6.3.2 Binarni ROC



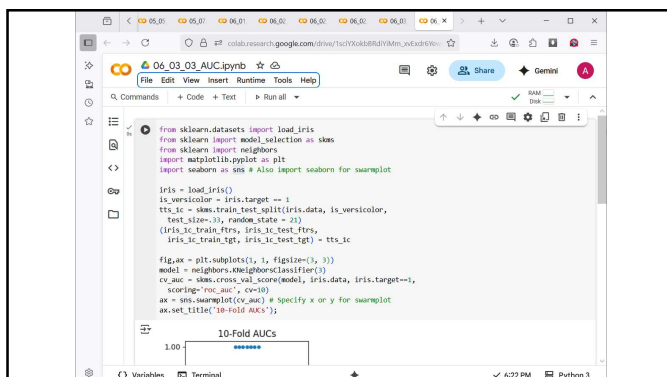
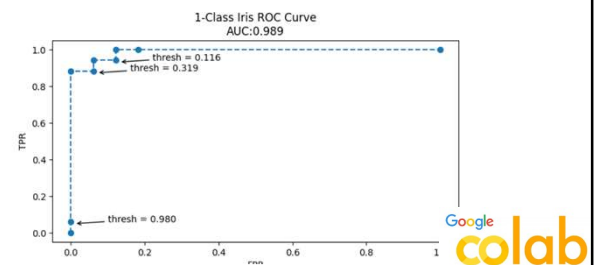
Google
colab

```
[24]: from sklearn.datasets import load_iris
from sklearn import model_selection as skms
from sklearn import naive_bayes
from sklearn import metrics
import matplotlib.pyplot as plt
import numpy as np

iris = load_iris()
is_vericolor = iris.target == 1
tts_ic = skms.train_test_split(iris.data, is_vericolor,
test_size=0.3, random_state=21)
(iris_ic_train_ftrs, iris_ic_test_ftrs,
iris_ic_train_tgt, iris_ic_test_tgt) = tts_ic
# build, fit, predict (probability scores) for NB model
gnb = naive_bayes.GaussianNB()
prob_true = (gnb.fit(iris_ic_train_ftrs, iris_ic_train_tgt)
.predict_proba(iris_ic_test_ftrs[:, 1]) * [1, 1]) >= prob_true)
print(prob_true)
```



6.3.3 AUC: Površina ispod (ROC) krivulje



6.3.4. Višeklasni modeli, One-versus-Rest i ROC

```

from sklearn.datasets import load_iris
from sklearn import model_selection as skms
from sklearn import neighbors
from sklearn import preprocessing
import sklearn.preprocessing and assign it the alias skpre
import matplotlib.pyplot as plt
import seaborn as sns # also import seaborn for heatmap
iris = load_iris()

check_out = [0, 50, 100]
print("Original Encoding")
print(iris.target[check_out])
print("Multi-label Encoding")
print(skpre.LabelBinarizer(iris.target, classes=[0, 1, 2])[check_out])

Original Encoding
[0 0 0]
'Multi-label' Encoding
[[1 0 0]
 [0 1 0]
 [0 0 1]]

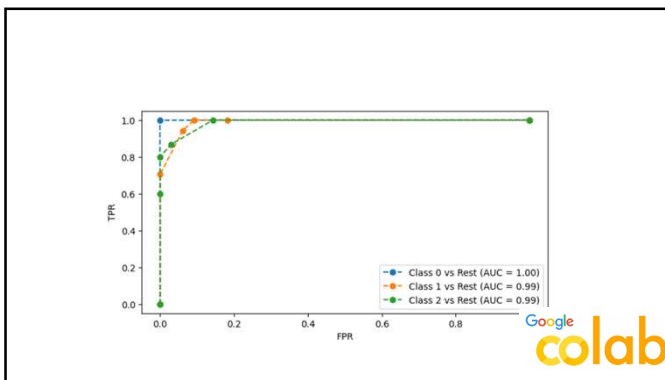
iris_multi_tgt = skpre.LabelBinarizer(iris.target, classes=[0, 1, 2])
(im_train_ftrs, im_test_ftrs,

```

```

iris_multi_tgt = skpre.LabelBinarizer(iris.target, classes=[0, 1, 2])
# im -> "iris multi"
(im_train_ftrs, im_test_ftrs,
 im_train_tgt, im_test_tgt) = skms.train_test_split(iris.data,
 iris_multi_tgt, test_size=.33, random_state=0)
# knn wrapped up in one-versus-rest (3 classifiers)
knn = neighbors.NeighborsClassifier(n_neighbors=5)
from sklearn.metrics import roc_auc_score
from sklearn.multiclass import OneVsRestClassifier
ovr_knn = OneVsRestClassifier(knn)
pred_probs = (ovr_knn.fit(im_train_ftrs, im_train_tgt)
 .predict_proba(im_test_ftrs))
# main ROC plots
lbl_fet = "Class () vs Rest (AUC = {:.2f})"
fig, ax = plt.subplots(figsize=(8, 4))
for cls in [0, 1, 2]:
    fpr, tpr, _ = metrics.roc_curve(im_test_tgt[:, cls],
    pred_probs[:, cls])
    label = lbl_fet.format(cls, metrics.auc(fpr, tpr))
    ax.plot(fpr, tpr, '-', label=label)
    ax.legend()
ax.set_xlabel("FPR")
ax.set_ylabel("TPR")

```



```

# Za svaki od 3 klasifikatora (klasa 0, 1, 2)
for i in range(3):
    print(f"Klasa {i}:")

    # Stvarna binarna klasa i predikcija vjerojatnosti za tu klasu
    y_true_bin = im_test_tgt[:, i]
    y_score = pred_probs[:, i]

    # Izračun ROC metrika
    fpr, tpr, thresholds = metrics.roc_curve(y_true_bin, y_score)

    # Ispis svih pragova i odgovarajućih TPR/FPR vrijednosti
    for j, thresh in enumerate(thresholds):
        print(f"Prag: {thresh:.2f} | TPR: {tpr[j]:.2f} | FPR: {fpr[j]:.2f}")

Klasa 0:
Prag: inf | TPR: 0.00 | FPR: 0.00
Prag: 1.00 | TPR: 1.00 | FPR: 0.00
Prag: 0.00 | TPR: 1.00 | FPR: 1.00

Klasa 1:
Prag: inf | TPR: 0.00 | FPR: 0.00
Prag: 0.00 | TPR: 1.00 | FPR: 1.00

```

KRAJ!